

WRITING A SPEC

Keep this next to you. You will use it every time you write one.

THE ONLY TEST THAT MATTERS

"What would the agent have to guess?"

Ask it of every line you write. If there is an answer, that is a gap, and the agent will fill it with something plausible and confident.

THE SIX PARTS

Why. The problem, in two lines. This is not decoration. It is what lets the agent make a sensible call on anything you did not spell out.

What. The behaviour, with real numbers and the edges. This is most of the spec.

Context. What already exists that this has to fit into. The files it will touch, one pattern it must match with the file that shows it, and the decisions already settled so nobody reopens them.

Constraints. What it must not do, and what is out of scope. The part almost everyone leaves out, and the part that saves you most.

Tasks. Numbered, small, each saying what to build, which files it touches, and how you will know it is done.

Done. One check for the whole feature, once every task is finished. Not the same as the verify lines, which only tell you each piece works on its own.

A spec is a file. Save it in the project, commit it before you write any code, and keep it with the work. A spec that lives in a chat window cannot be reviewed by anyone else, cannot be picked up tomorrow, and cannot be pointed at when somebody asks why the feature does what it does.

THE MAIN SKILL: VAGUE TO PRECISE

Nearly all of spec writing is this move. The left column is what people write first. The right is the same thing with nothing left to guess.

VAGUE

PRECISE

"Close old tickets."

"Close any ticket with no customer reply for 14 days. Exactly 14 days counts, 13 does not."

"Mark it closed."

"Set status to Closed and record who closed it as 'system', not as a person."

"Handle errors sensibly."

"A ticket that is already closed is skipped and reported, not treated as an error."

"Don't close important ones."

"Never auto-close a ticket flagged Urgent, whatever its age."

"Follow the existing patterns."

"Match the way TicketService returns errors, as in TicketService.close()."

"Add tests."

"Verify: a 14 day old ticket closes, a 13 day old one does not, an Urgent one is untouched."

If a line contains *old*, *recent*, *large*, *sensibly*, *appropriately*, or *properly*, it is not finished. Those words feel like requirements and are actually gaps.

CONTEXT: WHAT THE AGENT CANNOT SEE

An agent reads what you give it. It does not know which of the forty files matter, which pattern in the codebase is the one you actually follow, or which arguments were settled last week. Left to itself it will invent a new way of doing something you already do.

Three things, and no more:

Files: TicketService, Ticket, TicketService tests
Pattern: errors are returned, not thrown. See TicketService.close()
Settled: no new libraries; ticket state stays in the Ticket record

Write it for somebody who has never seen the project, because that is exactly what a fresh session is.

CONSTRAINTS: THE EAGER AGENT

An agent wants to be helpful, so left alone it will add things you never asked for. Commonly: notifications and emails, a screen to manage the thing, a database table, a new library, logging, an admin override.

If you do not want it, say so. Out of scope is not padding, it is the section doing the most work.

Out of scope: emailing the customer, any UI, changing the reply deadline, and anything that touches billing.

TASKS

Small enough to review in one sitting. That is the whole rule. If you could not read it and say whether it is right, it is too big, and you will end up approving something you do not understand.

A second rule that gives you the same answer: each task should be safe to commit on its own. Things that must ship together are one task. Things that do not are two.

Each task says three things:

2. Apply the close.

Touches: TicketService

Verify: a ticket last replied to 14 days ago becomes Closed;

one at 13 days is unchanged.

"Verify: it works" is not a verify line. If there are no real values in it, you have not written one.

HOW BIG SHOULD THE SPEC BE

Match it to the job. A spec that takes longer to write than the work it describes is the wrong size.

THE JOB

THE SPEC

A bug fix

Why, What, one task. Skip the rest.

A small feature, a few files

All six parts, one page, two or three tasks.

Anything touching ten files or more

Two specs. It is more than one piece of work.

HOW TO ACTUALLY WRITE ONE

You do not start from a blank page.

- 01 Describe the feature to Claude** and ask it to draft the spec, and to ask you questions rather than choosing for you.
- 02 Answer its questions yourself.** Those questions are the decisions. Do not hand them back.
- 03 Read the whole draft.** Change anything it decided that you did not.

- 04 **Check it against five things.** Are the numbers ones you chose? Does the context name real files? Is the out-of-scope list doing any work? Could you review each task? Does every verify line have real values in it?
- 05 **Commit the spec, before any code.** The first commit in the history should be the decision, not the result.

The value is not in the draft. It is in what you changed and what you confirmed.

THEN RUN IT, ONE TASK AT A TIME

For each task, in order:

- 01 **One task only.** "Read the spec and implement task 1 only. Do not start task 2."
- 02 **Verify it** against that task's own verify line, using the actual values. Then the boundary case. Then decide whether you would put your name on it.
- 03 **Read the diff.** Not the summary Claude gives you. The diff.
- 04 **Commit it on its own**, with the task in the message: T1: apply the close .
- 05 **Start the next task in a fresh session**, with the spec and the code as it now stands. A long conversation slowly fills up with earlier drafts, abandoned ideas and things you corrected once, and the agent treats all of it as true.

If the work proves the spec wrong, change the spec. You will find things while building that you could not have known while writing. That is normal, and it is not a failure of the spec. Update the file and commit it, so the plan and the code stay in step. Telling the chat is not updating the spec.

THE FOUR STEPS THIS SITS INSIDE

Frame write the spec. **Direct** one task at a time. **Verify** check it against the verify line. **Own** be able to explain every line you keep.